APK is the installation package format used by Android applications. It contains the compiled application code, configuration files, and resources required for execution. Decompilation is needed for security auditing because an APK is not directly readable as normal source code. By decompiling the package, we can inspect the app structure, identify important components, and review security-relevant logic such as authentication and session handling.

We used AI assistance to select and set up a suitable decompilation tool. Based on its ability to directly open APK files, display decompiled Java code, and expose both the manifest and application classes in a readable structure, JADX was chosen as the primary analysis tool. The AI-assisted tool selection process was recorded separately in the AI usage log.

The APK was successfully opened in JADX. We were able to access the following required components:

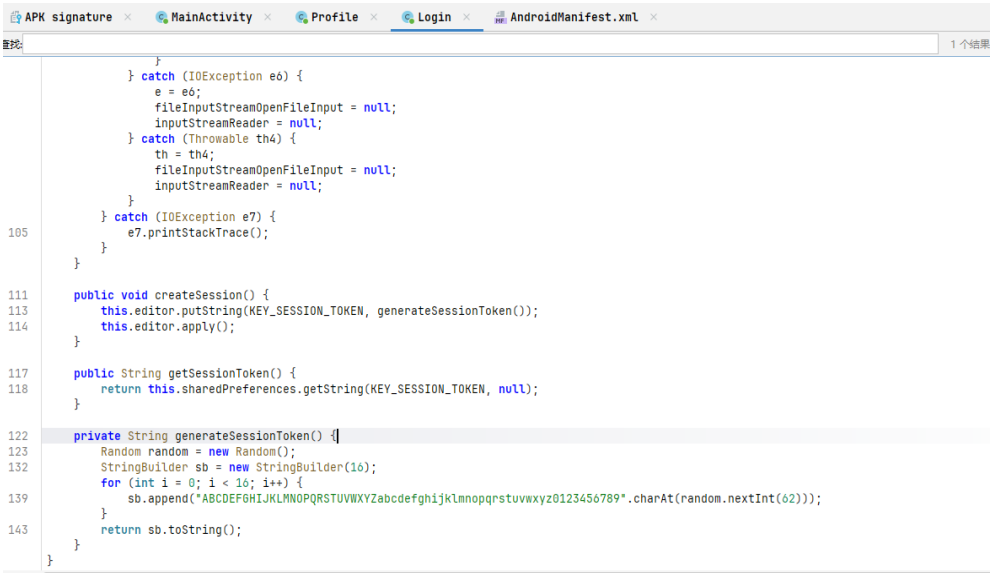
Package name: com.example. mastg_test0016

Main activity: com.example. mastg_test0016.MainActivity

Manifest: AndroidManifest.xml

Login-related class: com.example. mastg_test0016.Login

A screenshot of the decompiled **Login** class is included as evidence of successful code recovery and early inspection of the authentication workflow.



2.1 System Overview

The application facilitates a local authentication and session management workflow. As illustrated in Figure 2, the system architecture consists of three primary activities: **MainActivity** for credential entry, Login for authentication, and Profile for authorized user access.

2.2 System Model and Data Flow

The system model integrates UI components with local storage mechanisms.

Key Assets: The identified high-value assets include user credentials (usernames and passwords) and the security-sensitive **sessionToken**.

Data Flow: As depicted in Figure 2, user input flows from **MainActivity** to the local credentials.txt file. During authentication, Login retrieves these credentials for validation. Upon success, it generates a session token stored in **SharedPreferences**, which is subsequently verified by the Profile activity to grant access.

2.3 Threat Model

To align with the assessment criteria, we define a realistic threat environment:

Attacker Profile: We assume a local adversary with filesystem access, such as a user on a rooted device or an entity with unauthorized ADB access.

Attacker Goals and Linkage: The attacker's primary objectives are:

1. Credential Theft: Exploiting the insecure plaintext storage in credentials.txt to exfiltrate passwords.
2. Session Hijacking: Performing a Session Prediction Attack by leveraging the predictable nature of the weak PRNG (pseudo-random number generator) identified in the session token generation logic.

2.4 **Core assumptions:**

We assume that the decompiled code reflects the main authentication logic of the app. We also assume that an attacker can reverse engineer the APK, inspect how credentials and session data are handled, and observe when session values are

generated.

Attacker goals.: The attacker aims to obtain unauthorised access to the authenticated state by predicting, reproducing, or otherwise abusing the session token used by the app. A successful attack would allow the attacker to impersonate a legitimate user or bypass the intended login flow.

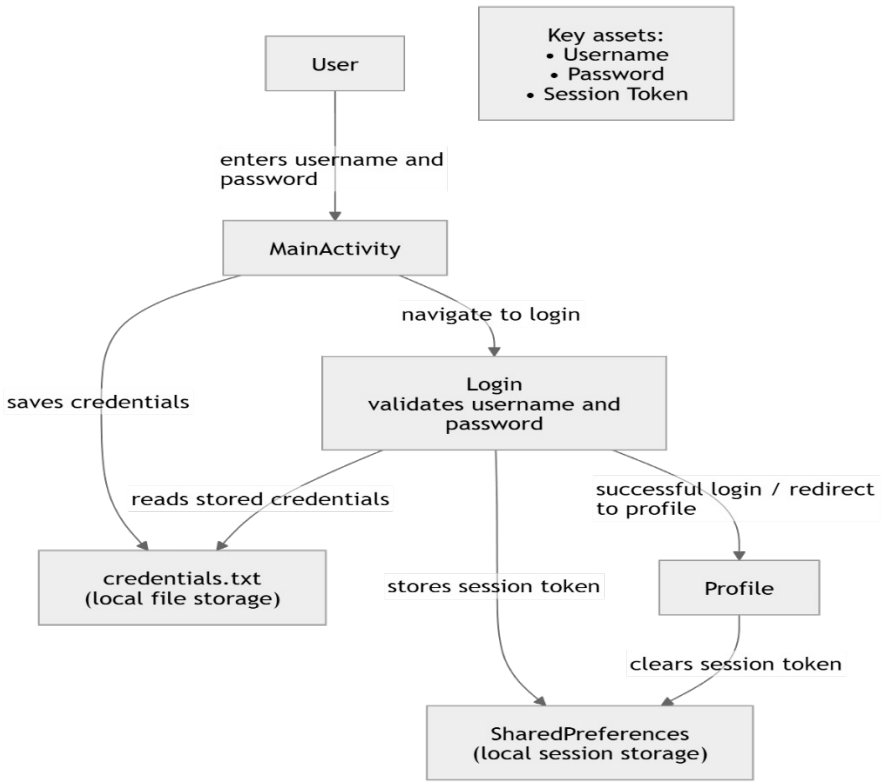


Figure 2: Application System and Data Flow Model

3.1 Discovery Methodology

We utilized **JADX-GUI** for static analysis to inspect the decompiled source code. Our discovery process was supported by an **AI-assisted workflow**. We prompted an LLM to identify security-sensitive functions related to "randomness," "tokens," and "credential storage" within the com.example.mastg_test0016 package.

AI Interaction Summary: * Prompt: "Analyze the generateSessionToken method in the provided code. Is the random number generator cryptographically secure for authentication tokens?"

- **Validation:** The AI correctly flagged the use of java.util.Random as a predictable PRNG, which we manually verified against the decompiled Login.java.

3.2 Identified Vulnerabilities

Following the assignment scope, we focused on vulnerabilities related to randomness and cryptographic misuse

Class	**Method**	**Generator API / Helper**	**Security Role**
Login	generateSessionToken()	java.util.Random	Authentication Token (Session Management)
MainActivity	saveCredentialsToFile()	openFileOutput (Plaintext)	Credential Persistence (Cryptographic Misuse)

3.3 Vulnerability Assessment

The use of java.util.Random in Login.java is unsuitable for generating session tokens. Unlike java.security.SecureRandom, java.util.Random uses a Linear Congruential Generator (LCG) with a state space of only 2^{48}. If an attacker observes a sequence of generated tokens, they can determine the internal state and predict all future tokens. Furthermore, the lack of encryption in MainActivity.java violates basic cryptographic principles for storing sensitive user credentials.

4.1 Technical Root Cause

The core vulnerability lies in the generateSessionToken() method within Login.java, which employs java.util.Random to produce 16-character authentication tokens. As a Linear Congruential Generator (LCG), java.util.Random is not cryptographically secure. Its internal state is only 48 bits, and the algorithm is entirely deterministic. An attacker who observes a small sequence of outputs can mathematically reconstruct the generator's internal state, allowing them to predict all future tokens with absolute certainty.

4.2 Attacker Model

We define a **Local Adversary** or a **Privileged Observer** as the primary threat.

- **Capabilities:** The attacker has the ability to monitor local storage (e.g., via a

rooted device) or intercept the application's internal intent broadcasts.

- **Knowledge:** The attacker requires access to at least two to three valid session tokens generated by the target device to synchronize with the PRNG state.

4.3 Step-by-Step Attack Scenario: Session Prediction

The following scenario demonstrates how the weak randomness leads to a complete authentication bypass:

1. **Observation:** The attacker creates several accounts or performs multiple logins to collect a series of valid sessionToken values from the SessionPrefs SharedPreferences file.
2. **State Reconstruction:** Using a PRNG state recovery tool (e.g., a simple script utilizing the LCG parameters), the attacker calculates the current 48-bit seed of the java.util.Random instance.
3. **Token Prediction:** The attacker computes the next token that the system will generate for the legitimate victim's upcoming login.
4. **Impersonation:** The attacker manually injects the predicted token into their own device's SessionPrefs or intercepts the victim's session.
5. **Unauthorized Access:** The attacker launches the Profile activity. The application validates the predicted token and grants full access to the victim’s profile without requiring valid credentials.

5.1 Remediation for Insecure Randomness

To mitigate the session prediction vulnerability, the java.util.Random instance in Login.java must be replaced with java.security.SecureRandom.

- **Implementation:** The code should be updated to SecureRandom random = new SecureRandom();.
- **Why it works:** Unlike the linear congruential generator used by java.util.Random, SecureRandom produces non-deterministic, cryptographically strong sequences that comply with FIPS 140-2 standards. Its state space is significantly larger, making it computationally infeasible for an attacker to predict future tokens even if previous ones are known.

5.2 Remediation for Plaintext Storage

To protect user credentials in MainActivity.java, the application must move away from plaintext storage.

- **Implementation:** Instead of storing raw passwords, the app should use a strong cryptographic hashing algorithm like **Argon2** or **PBKDF2** with a unique salt for each user. Alternatively, sensitive data can be encrypted using the **Android Keystore System**, which provides hardware-backed security for cryptographic keys.
- **Why it works:** These measures ensure that even if the credentials.txt file is exfiltrated from a rooted device, the actual passwords remain unreadable and protected against rainbow table or brute-force attacks.